

# Aula 05: Memória Cache

## ACH2055 – Organização e Arquitetura de Computadores II

Arquitetura de Computadores: uma abordagem quantitativa. Patterson e Hennessy, 5ª edição – Apêndice B  
Arquitetura e Organização de Computadores. William Stallings, 8ª edição – Capítulo 4.

Valdinei Freire da Silva

Escola de Artes, Ciências e Humanidades - USP

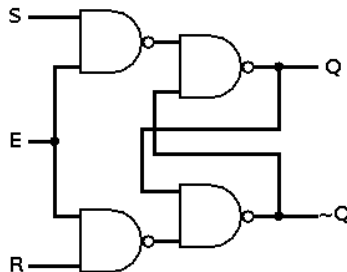
2025

# Flip-Flops

## FLIP-FLOP RS com *enable*

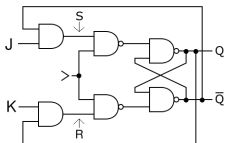
- ▶ E (*enable*) - habilita o flip-flop
- ▶ S (*set*) - habilita a saída do flip-flop
- ▶ R (*reset*) - desabilita a saída do flip-flop

| E | S | R | Q                          |
|---|---|---|----------------------------|
| 0 | X | X | Q                          |
| 1 | 0 | 0 | Q                          |
| 1 | 1 | 0 | 1                          |
| 1 | 0 | 1 | 0                          |
| 1 | 1 | 1 | proibido ( $Q = \bar{Q}$ ) |

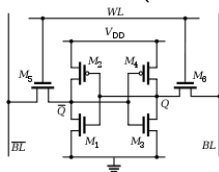


# Tecnologia de Memória

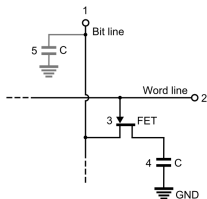
## Digital Flip-Flop



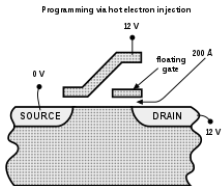
## Static RAM (SRAM)



## Dynamic RAM (DRAM)

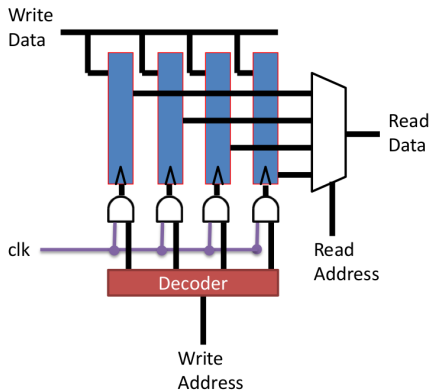


## Flash Memory

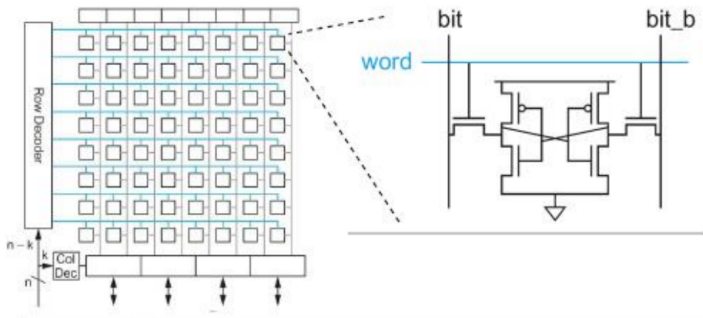


# Banco de Registradores

## Solução Ingênua



## Matrix de Registradores



# Hierarquia de Memória

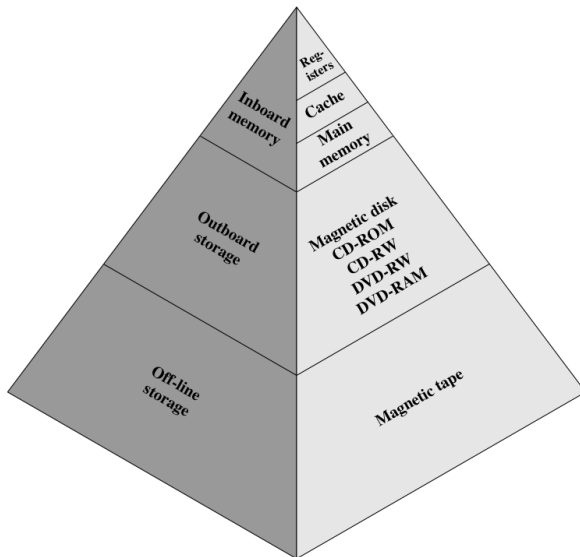


Figure 4.1 The Memory Hierarchy

# Hierarquia de Memória

## Características

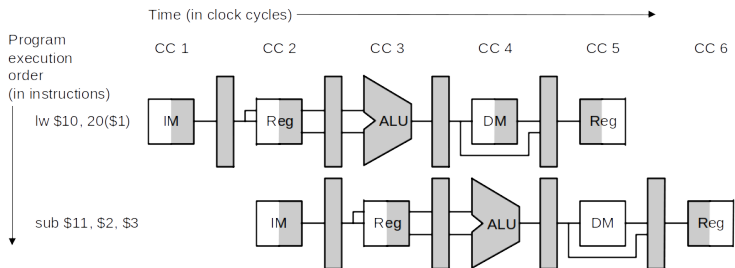
- ▶ Capacidade de Armazenamento
- ▶ Latência
- ▶ Largura de Banda

Compromisso: tamanho do chip e tempo de ciclo

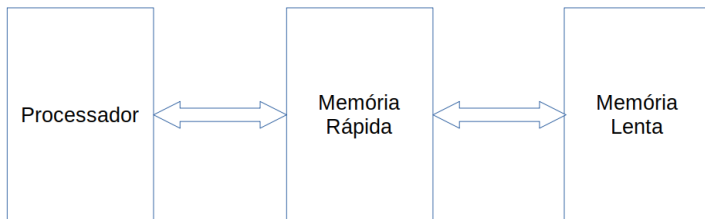
Hierarquia de Memória: garantir armazenamento arbitrário com tempo de ciclo baixo

# Hierarquia de Memória

## Pipeline



# Memória Cache

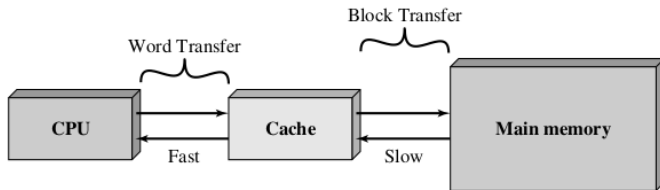


Operação:

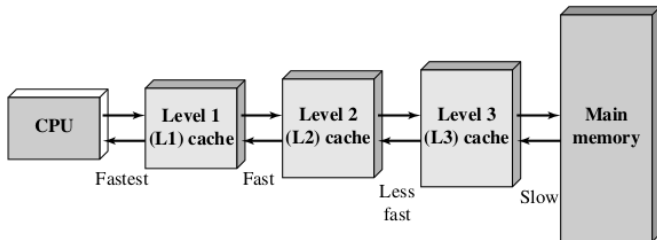
- ▶ Se palavra está na memória rápida, latência de 1 ciclo
- ▶ Se palavra está na memória lenta, latência maior que um ciclo

Memória Cache: palavras na memória rápida são escolhidas de forma dinâmica

# Níveis de Cache



(a) Single cache



(b) Three-level cache organization

# Memória Cache

Hierarquia de Memórias funciona apenas se a maioria dos dados lidos/escritos estão na memória rápida.

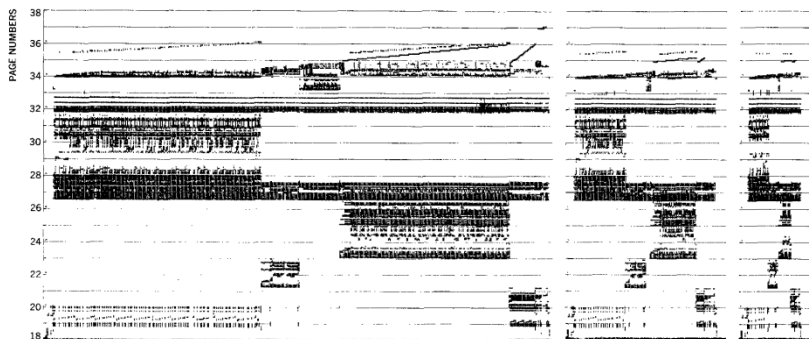
Princípios:

- ▶ **Localidade Temporal:** se um local é referenciado, é provável que ele será referenciado novamente num futuro próximo
  - ▶ Exemplo: loops, variáveis locais
- ▶ **Localidade Espacial:** se um local é referenciado, é provável que locais perto dele sejam referenciado num futuro próximo
  - ▶ Exemplo: busca de instruções, leitura de vetores

Estratégia:

- ▶ explorar localidade temporal, lembrando os conteúdos acessados recentemente
- ▶ explorar localidade espacial, buscando blocos de dados em volta de locais acessados recentemente

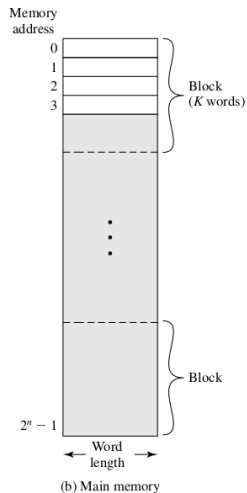
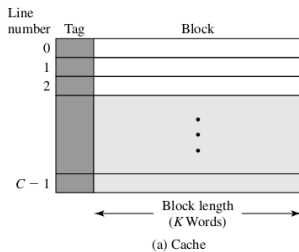
# Memória Cache



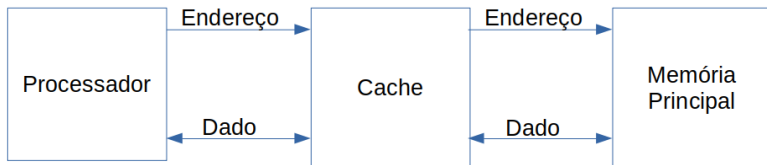
Exemplo real de acesso a memória<sup>1</sup>

<sup>1</sup>Donald J. Hatfield, Jeanette Gerald. Program Restructuring for Virtual Memory. IBM Systems Journal 10(3): 168-192 (1971)]

# Memória Cache



# Memória Cache



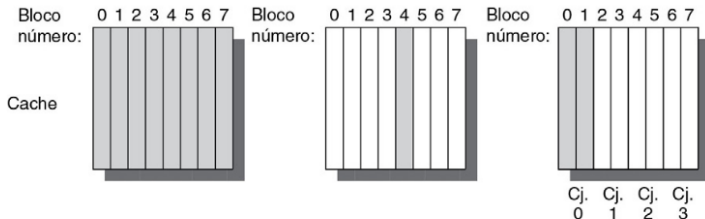
- ▶ P1: Onde um bloco pode ser colocado no nível superior? (posicionamento de bloco)
- ▶ P2: Como um bloco é localizado se estiver no nível superior? (identificação de bloco)
- ▶ P3: Qual bloco deverá ser substituído em caso de falta (cache miss)? (substituição de bloco)
- ▶ P4: O que acontece em uma escrita? (estratégia de escrita)

# Posicionamento de Bloco

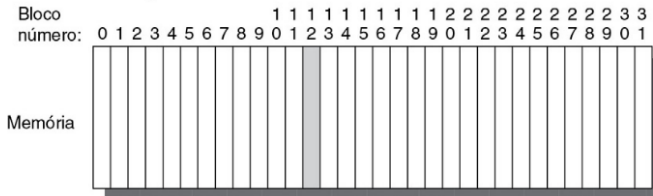
Totalmente associativo:  
bloco 12 pode entrar  
em qualquer lugar

Mapeamento direto:  
bloco 12 só pode ir  
para o bloco 4  
( $12 \bmod 8$ )

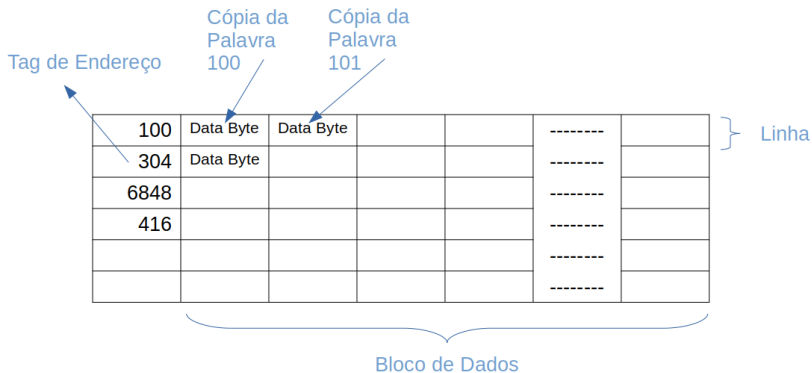
Associativo em conjunto:  
bloco 12 pode ir para  
qualquer lugar no conjunto 0  
( $12 \bmod 4$ )



Endereço de frame de bloco

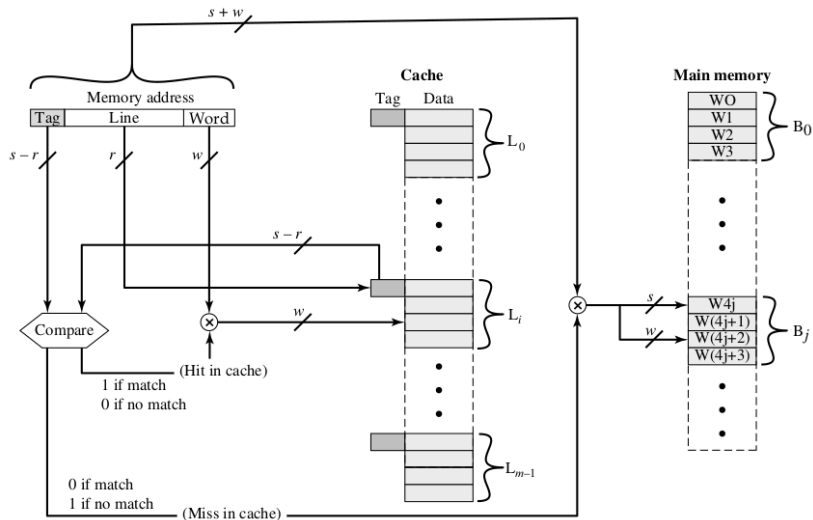


# Identificação de Bloco

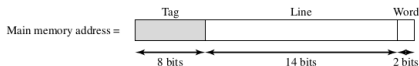
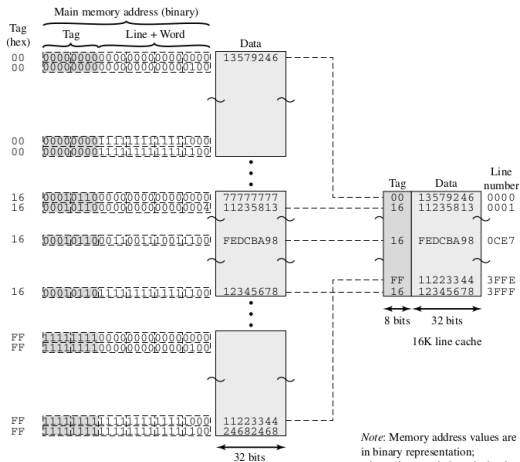


- ▶ Endereço: Tag + Word
- ▶ Endereço: Tag + Line + Word

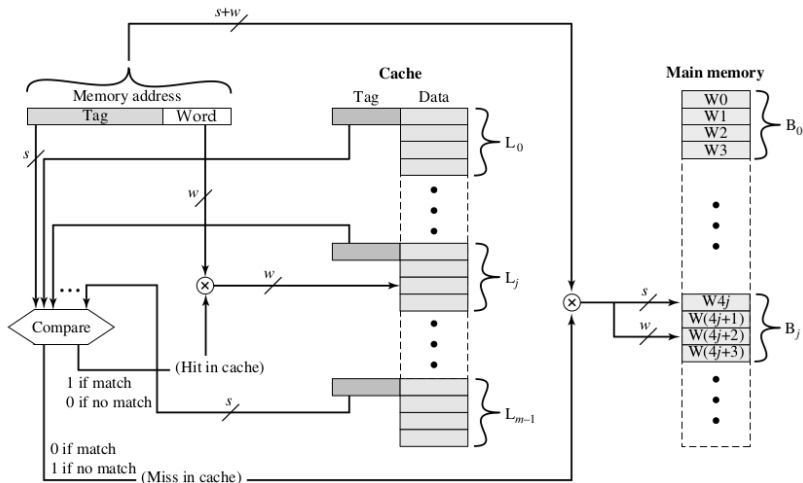
# Exemplo: Mapeamento Direto



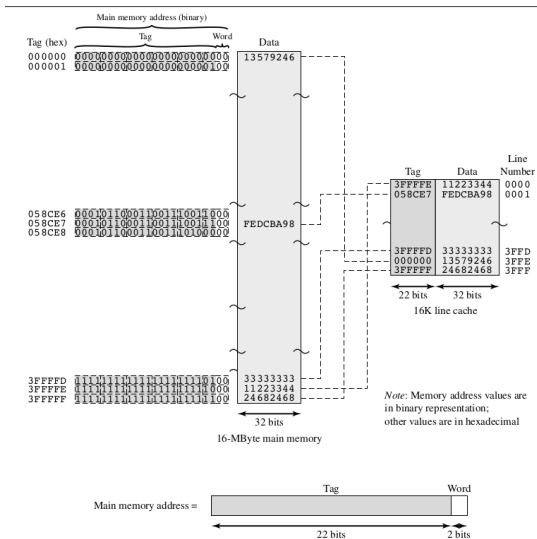
# Exemplo: Mapeamento Direto



# Exemplo: Totalmente Associativo



# Exemplo: Totalmente Associativo



- ▶ Nenhuma escolha em mapeamento direto
- ▶ Em um cache associativo, qual bloco de um conjunto deve ser removido quando o conjunto está cheio?
  - ▶ Aleatório
  - ▶ Least Recently Used (LRU)
    - ▶ ordenação das linhas deve ser mantida após cada acesso
    - ▶ implementação real apenas factível em conjuntos pequenos (2 vias)
  - ▶ First In, First Out (FIFO)
    - ▶ usado em caches associativos com muitas vias
  - ▶ Not Most Recently Used (NMRU)
    - ▶ alternativa barata ao LRU

## ▶ Cache Hit

- ▶ Write Through: escreve na cache e na memória principal
- ▶ Write Back: escreve apenas na cache, memória principal é escrita apenas quando bloco é removido

## ▶ Cache Miss

- ▶ No Write Allocate: escreve apenas na memória principal
- ▶ Write Allocate: copia o bloco para a cache, então escreva

# Tipos de Cache Miss

- ▶ Compulsória: O primeiro acesso a um bloco não pode ser feito na cache, por isso o bloco precisa ser trazido para a cache.
- ▶ Capacidade: Se a cache não puder conter todos os blocos necessários durante a execução de um programa, as faltas de capacidade (além das faltas compulsórias) ocorrerão por causa dos blocos sendo descartados e recuperados mais tarde.
- ▶ Conflito (colisão): Se a estratégia de colocação do bloco é a associatividade em conjunto ou o mapeamento direto, faltas por conflito (além das faltas compulsórias e por capacidade) ocorrerão porque um bloco pode ser descartado e mais tarde recuperado se muitos blocos forem mapeados para o seu conjunto.

Tempo médio de acesso à memória = Tempo de acerto + Taxa de falta  $\times$  Penalidade de falta

- ▶ Reduzir a taxa de falta: tamanho de bloco maior, tamanho de cache maior e associatividade mais alta
- ▶ Reduzir a penalidade por falta: caches multiníveis
- ▶ Reduzir o tempo para acerto na cache: associatividade mais baixa